

Lisa Thomas

I.I.S. "G. Vallauri" — Classe 5B Informatica

Anno Scolastico 2025/2026

DiceRoll

Applicazione web per la gestione della scheda del personaggio di D&D 5e, con multiplayer locale via WebSocket.

Indice

1. Introduzione
 - 1.1 Cos'è Dungeons & Dragons
 - 1.2 Il progetto
 - 1.3 Obiettivi del progetto
2. Lato Client
 - 2.1 Electron — dal browser al desktop
 - 2.2 HTML5, CSS3 e Bootstrap
 - 2.3 JavaScript — logica e interattività
 - 2.4 Salvataggio dei dati
 - 2.5 Internazionalizzazione (i18n)
3. Lato Server
 - 3.1 Python e asyncio
 - 3.2 Il protocollo WebSocket
 - 3.3 Architettura delle stanze
4. Funzionalità principali
 - 4.1 Scheda del personaggio e combattimento
 - 4.2 Mappe e marcatori
 - 4.3 Pannello Dungeon Master
 - 4.4 Party Chat
5. Sviluppo e strumenti
 - 5.1 Git e controllo versione
 - 5.2 Electron Forge
 - 5.3 Difficoltà riscontrate
6. Tecnologie utilizzate
 - 6.1 Motivazione dietro alle tecnologie utilizzate
7. Conclusioni
 - 7.1 Possibili Sviluppi Futuri

1. Introduzione

1.1 Cos'è Dungeons & Dragons

Dungeons & Dragons (D&D) è il gioco di ruolo da tavolo più celebre al mondo, pubblicato nel 1974 e oggi alla sua quinta edizione. Un gruppo di giocatori interpreta personaggi fantasy all'interno di un mondo immaginario guidato dal Dungeon Master (DM), figura che narra la storia e arbitra le regole. L'esito delle azioni viene determinato tirando dadi poliedrici — il più iconico è il d20. Ogni personaggio è definito da una scheda cartacea che raccoglie caratteristiche, abilità, equipaggiamento, incantesimi e Punti Ferita: aggiornarla e consultarla durante il gioco può risultare laborioso, ed è proprio qui che nasce l'esigenza di DiceRoll.

1.2 Il progetto

DiceRoll è un'applicazione web desktop-only sviluppata per digitalizzare e arricchire l'esperienza di gioco di Dungeons & Dragons 5ª Edizione. Il progetto nasce dall'esigenza di avere uno strumento interattivo che sostituisca la classica scheda cartacea, eliminando problemi legati a calcoli manuali, spazio fisico e leggibilità.

L'idea iniziale di una semplice scheda digitale si è evoluta rapidamente includendo funzionalità avanzate come il caricamento di mappe personalizzate con marcatori interattivi, un pannello dedicato al Dungeon Master e un sistema di Party Chat in tempo reale che consente ai giocatori sulla stessa rete di comunicare direttamente dall'applicazione.

Dal punto di vista tecnico, DiceRoll è composta da un frontend statico in HTML, CSS e JavaScript distribuito tramite Electron, e da un server Python che gestisce la comunicazione WebSocket tra i giocatori. L'applicazione è open source e progettata per essere utilizzata senza configurazioni complesse: è sufficiente avviare l'eseguibile.

Una delle principali scelte progettuali è stata quella di realizzare un'applicazione completamente offline. Il salvataggio delle schede avviene tramite la File System Access API direttamente sul filesystem locale, mentre il backup automatico viene conservato nel localStorage di Electron. Anche la Party Chat utilizza un server WebSocket eseguito sulla rete locale dell'utente, senza alcun transito di dati su Internet. Questo approccio garantisce privacy, indipendenza da servizi cloud e la possibilità di utilizzare DiceRoll in qualsiasi contesto.

1.3 Obiettivi del progetto

Gli obiettivi principali che hanno guidato lo sviluppo di DiceRoll possono essere riassunti in tre aree:

- **Usabilità:** l'applicazione deve essere immediata da usare anche per chi non ha dimestichezza con strumenti digitali complessi. La scheda deve essere navigabile a tab, con calcoli automatici e feedback visivo chiaro;
- **Completezza:** coprire tutte le meccaniche ufficiali di D&D 5e, più alcune estensioni homebrew come lo Scaling delle Abilità, senza sacrificare la leggibilità dell'interfaccia;
- **Portabilità:** il prodotto finale deve girare su qualsiasi PC Windows senza richiedere l'installazione di software aggiuntivi (eccetto Python per il modulo server, che è opzionale).

SCHEDA PERSONAGGIO EQUIPAGGIAMENTO INCANTESIMI MAPPE SCALING ABILITA' DUNGEON MASTER PARTY

Reset Carica Salva Salva con nome

INFORMAZIONI PERSONAGGIO

NOME PERSONAGGIO: Jason Grace CLASSE: Chierico della Tempesta RAZZA: Genasi dell'Aria BACKGROUND: Soldato ALLINEAMENTO: NB LIVELLO: 16

ESPERIENZA: 0 ETA': 17 ALTEZZA: 1.80m PESO: 80kg CABINAGIONE: Chiara CAPELLI: Biondi e corti OCCHI: Grigio nuvola

CARATTERISTICHE

FOR	16	+3
DES	18	+4
COS	14	+2
INT	10	+0
SAG	20	+5
CAR	16	+3

TIRI SALVEZZA

FOR		+3
DES	☒	+10
COS		+2
INT		+0
SAG	☒	+11
CAR		+3

ABILITA'

NOME	MOD	C	M
Acrobazia (DES)	+4		
Addentrare Animali (SAG)	+5		
Arcano (INT)	+12		☒
Atletica (FOR)	+9	☒	
Furtivita' (DES)	+4		
Indagare (INT)	+0		
Ingianno (CAR)	+3		
Intimidire (CAR)	+9	☒	
Intrattenere (CAR)	+3		
Intuizione (SAG)	+8		
Medicina (SAG)	+5		
Natura (INT)	+6	☒	
Percezione (SAG)	+11	☒	
Persuasione (CAR)	+3		
Rapidita' di mano (DES)	+4		
Religione (INT)	+0		
Sopravvivenza (SAG)	+11	☒	
Storia (INT)	+0		

INIZIATIVA +4 **VELOCITA'** 30 **COMPETENZA** 6 **PERCEZ. PASS.** 21

PUNTI FERITA

ATTUALI	MASSIMI	TEMP
95	95	0

CA 18 CA TEMP 0 DADI VITA 16 16 d8

TS CONTRO MORTE

SUCCESSI ☐ ☐ ☐ FALLIMENTI ☐ ☐ ☐

ARMI

Arma	Bonus	Danno
fulgur	+10	4d8+4 + 20% 2d10 (Electric)
arma	+0	1d6
arma	+0	1d6
arma	+0	1d6
arma	+0	1d6
arma	+0	1d6
arma	+0	1d6
arma	+0	1d6

IMMAGINE (1/1)

Carica Rimuovi Prec Succ



Fig. 1 — Schermata principale della scheda del personaggio in DiceRoll con statistiche e modificatori.

Scheda D&D 5e

File Edit View Window Help

SCHEDA PERSONAGGIO EQUIPAGGIAMENTO INCANTESIMI MAPPE SCALING ABILITA' DUNGEON MASTER PARTY

Reset Carica Salva Salva con nome

MONETE

MR	MA	ME	MD	MP
3	20	0	10	0

EQUIPAGGIAMENTO (150 SLOT)

Nome	Slot	Descrizione
Dracma Oro	5	Comunicazione
Dracma Platino	2	Comunicazione
Dracma Rame	13	Comunicazione
Spilla di Giove	1	Quest
Oggetto 5	1	Note
Oggetto 6	1	Note
Oggetto 7	1	Note
Oggetto 8	1	Note
Oggetto 9	1	Note
Oggetto 10	1	Note
Oggetto 11	1	Note
Oggetto 12	1	Note
Oggetto 13	1	Note
Oggetto 14	1	Note
Oggetto 15	1	Note
Oggetto 16	1	Note

TRATTI & CARATTERISTICHE

Immune a danni elettrici (<06) e resistente a danni elettrici (<50%)
Danno aumentato quando sotto terra
Capacità di comprensione linguaggio animale (rapaci)
Controllo sulle tempeste di fulmini

APPUNTI GIOCATORE

Raggiungere il Nuova Roma per informare tutti i Pretori della situazione riguardante le identità di alcune divinità e discutere il piano d'azione

Restituire ad [Annabeth](#) la Falxa Egida (Quest finita)

Fig. 2 — Pannello dell'equipaggiamento e degli appunti del giocatore relativi al personaggio e alla campagna.

2. Lato Client

Il lato client di DiceRoll è costituito da tre file principali — `index.html`, `style.css` e `script.js` — più i dizionari di traduzione e le risorse statiche. Tutto il codice gira nel browser (o nel renderer process di Electron) senza dipendenze da framework pesanti: non viene usato React, Vue o Angular. Questa scelta è stata deliberata, per mantenere il progetto comprensibile e per approfondire la conoscenza del JavaScript nativo.

2.1 Electron — dal browser al desktop

Electron è un framework open source, sviluppato originariamente da GitHub e oggi mantenuto dalla community con il supporto di Microsoft, che consente di creare applicazioni desktop multiplatforma a partire da tecnologie web standard. Il suo funzionamento si basa sull'integrazione di due componenti: Chromium, il motore di rendering che esegue il codice HTML/CSS/JS, e Node.js, che fornisce l'accesso alle API di sistema come il filesystem, la rete e il processo principale.

Un'applicazione Electron si divide in due processi distinti. Il main process, definito nel file `main.js`, è responsabile della creazione della finestra dell'applicazione, della gestione del ciclo di vita del programma e dell'accesso alle API native. Il renderer process, invece, è sostanzialmente un browser Chrome che esegue il frontend: `index.html` con tutto il suo CSS e JavaScript.

In DiceRoll, Electron serve principalmente per un motivo: permettere all'utente finale di avviare l'applicazione con un doppio clic sull'eseguibile (`dice-roll.exe`), senza dover installare Node.js, aprire un terminale o configurare un server locale. Il packaging è gestito da Electron Forge tramite il file `forge.config.js`, che automatizza la compilazione e la creazione dell'installer per Windows.

 **In breve:** In sostanza, Electron permette di usare tecnologie web per costruire un'applicazione desktop a tutti gli effetti. La scelta è motivata dalla necessità che l'applicazione fosse fruibile da chiunque senza configurazioni: il risultato è un file eseguibile autonomo. Il lato negativo è una dimensione del pacchetto maggiore rispetto a un'applicazione nativa, ma per il contesto d'uso previsto si tratta di un compromesso accettabile.

2.2 HTML5, CSS3 e Bootstrap

HTML5 definisce l'intera struttura dell'interfaccia utente nel file `index.html`. L'architettura adottata è quella di una Single Page Application (SPA): tutto il markup viene caricato una sola volta al primo avvio, e la navigazione tra le diverse sezioni — Scheda, Equipaggiamento, Incantesimi, Mappe, Scaling, DM, Party — avviene esclusivamente tramite JavaScript, mostrando e nascondendo i pannelli corrispondenti senza mai ricaricare la pagina. Questo approccio garantisce una risposta immediata all'utente e un'esperienza fluida.

CSS3, nel file `style.css`, gestisce il tema visivo dell'applicazione. Uno degli aspetti più curati è il sistema di temi chiaro e scuro (Day Mode / Night Mode): le variabili CSS custom (`--bg-primary`, `--text-color`, ecc.) permettono di cambiare l'aspetto dell'intera interfaccia modificando un solo attributo sul tag `html`, senza dover riscrivere le regole di stile. La preferenza dell'utente viene salvata automaticamente in `localStorage` e ripristinata all'avvio successivo.

Bootstrap 5.3 è il framework CSS scelto per velocizzare lo sviluppo. Fornisce il sistema di griglia responsive, i componenti preconfezionati come le modali, i badge, le card e le navbar, oltre a una serie di classi di utilità per gestire margini, padding, display e colori.

L'utilizzo di Bootstrap ha permesso di concentrare lo sviluppo sulla logica applicativa, delegando al framework la coerenza visiva di base.

 **In breve:** Usare Bootstrap non è solo una scelta estetica: significa appoggiarsi a convenzioni di layout e accessibilità ampiamente consolidate, senza doverle ridefinire da zero. Il sistema di temi chiari/scuri tramite variabili CSS, invece, è una scelta che rende l'interfaccia facilmente personalizzabile: basta modificare poche variabili per cambiare l'aspetto dell'intera applicazione.

2.3 JavaScript — logica e interattività

Il file `script.js` contiene tutta la logica applicativa ed è il cuore di DiceRoll. È scritto interamente in JavaScript puro (Vanilla JS), senza l'ausilio di librerie come jQuery o framework reattivi come React. Questo ha richiesto una gestione manuale dello stato e degli aggiornamenti del DOM, ma ha anche permesso di comprendere a fondo i meccanismi sottostanti che normalmente vengono astratti dai framework.

Il codice fa largo uso delle funzionalità introdotte da ES6 e versioni successive: le arrow function per callback più concise, il destructuring per estrarre valori da oggetti e array, i template literal per la composizione dinamica di stringhe HTML, e `async/await` per gestire in modo leggibile le operazioni asincrone come la lettura e scrittura di file.

Le responsabilità principali di `script.js` sono:

- Mantenere in memoria lo stato completo della scheda del personaggio come oggetto JavaScript;
- Calcolare automaticamente modificatori delle caratteristiche, bonus di competenza, CD degli incantesimi, iniziativa e tutti gli altri valori derivati;
- Aggiornare il DOM in risposta agli input dell'utente, garantendo che i valori visualizzati siano sempre allineati con lo stato interno;
- Serializzare lo stato in JSON per il salvataggio su file e deserializzarlo al caricamento;
- Gestire la connessione WebSocket con il server per la Party Chat, inclusi riconnessione automatica e heartbeat.

 **In breve:** Non usare un framework reattivo ha richiesto più lavoro, ma ha avuto un valore formativo preciso: gestire manualmente l'aggiornamento dell'interfaccia e la sincronizzazione dei dati ha permesso di capire concretamente cosa fanno strumenti come React o Vue sotto la superficie, invece di usarli come scatole nere.

2.4 Salvataggio dei dati

DiceRoll implementa due strategie di salvataggio complementari, che coprono scenari d'uso differenti.

La prima è basata sulla File System Access API, un'API moderna dei browser (supportata anche in Electron) che permette di aprire, leggere e scrivere file direttamente nel filesystem dell'utente, in modo simile a come farebbe una normale applicazione desktop. Quando l'utente clicca 'Salva con nome', viene aperto il selettore file nativo del sistema operativo; il riferimento al file scelto viene mantenuto in memoria, così che i salvataggi successivi sovrascrivano direttamente lo stesso file senza richiedere ogni volta la scelta della destinazione. I dati vengono serializzati in formato JSON strutturato, che include tutte le informazioni della scheda e le relative impostazioni.

La seconda strategia è il salvataggio automatico su `localStorage`, che avviene ogni 60 secondi. È un meccanismo di protezione contro chiusure accidentali dell'applicazione: se la scheda viene chiusa senza salvare, al successivo avvio l'applicazione rileva il backup e

propone all'utente di ripristinarlo tramite una modale dedicata. Il backup viene ignorato se non contiene dati significativi, evitando prompt inutili all'apertura dell'applicazione.

💡 In breve: Le due modalità di salvataggio rispondono a esigenze diverse: il file JSON è la copia definitiva e portabile della scheda, mentre il backup automatico è una rete di sicurezza contro chiusure accidentali. Usarli insieme significa che i dati difficilmente vanno persi, anche se l'utente dimentica di salvare manualmente.

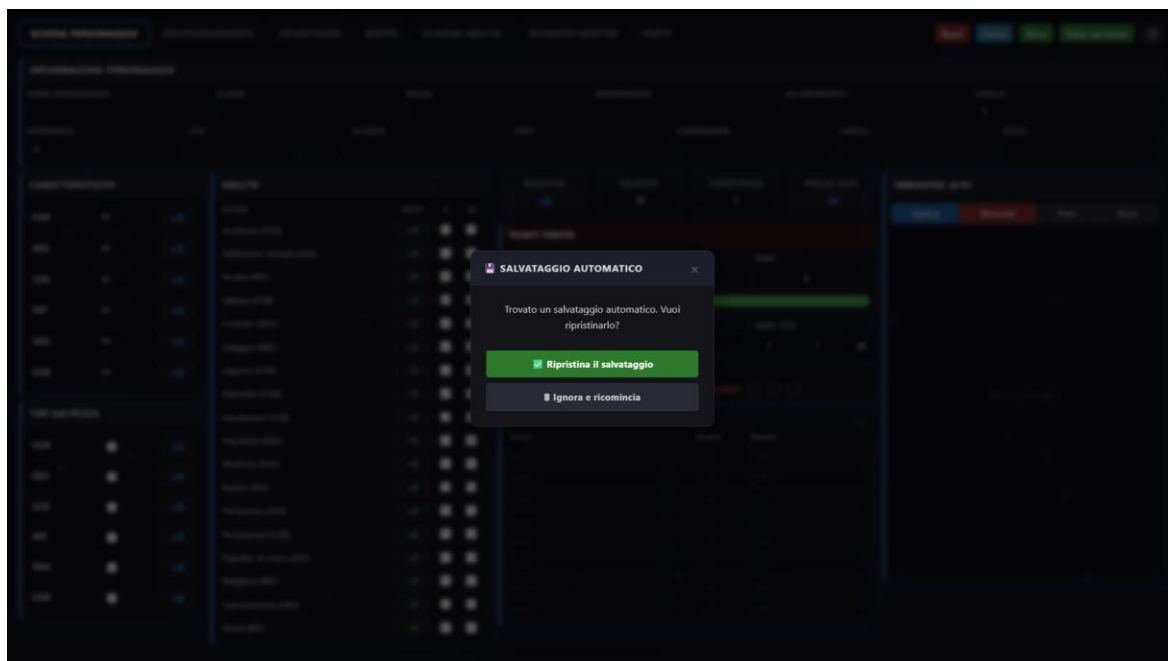


Fig. 3 — Finestra di dialogo per il salvataggio e il ripristino della scheda salvata solamente su localStorage in modo temporaneo.

2.5 Internazionalizzazione (i18n)

DiceRoll supporta due lingue — italiano e inglese — tramite un sistema di internazionalizzazione sviluppato da zero. I file ita.js ed eng.js espongono dizionari JavaScript organizzati in sezioni tematiche: tab di navigazione, pulsanti, messaggi di sistema, etichette dei campi della scheda, messaggi di errore e messaggi del server.

Il cambio lingua avviene in tempo reale senza ricaricare la pagina: ogni elemento dell'interfaccia che visualizza testo è marcato con un attributo data-i18n che punta alla chiave corrispondente nel dizionario. Quando l'utente seleziona una lingua nelle impostazioni, una funzione scorre tutti questi elementi e aggiorna il loro contenuto testuale in un'unica passata. La preferenza linguistica viene salvata automaticamente tra sessioni.

💡 In breve: Il sistema di internazionalizzazione è stato sviluppato internamente seguendo lo stesso principio dei framework professionali: i testi sono separati dal codice dell'interfaccia e organizzati in file dizionario. Questo significa che aggiungere una terza lingua richiederebbe solo un nuovo file, senza modificare nient'altro, e che la manutenzione delle traduzioni è indipendente dallo sviluppo dell'applicazione.

3. Lato Server

Il lato server di DiceRoll è composto da un unico file, `server.py`, che implementa il server WebSocket per la Party Chat. È una componente opzionale: l'applicazione funziona completamente anche senza server, che è necessario solo se si vuole usare la funzionalità multiplayer.

3.1 Python e asyncio

Python è un linguaggio di programmazione ad alto livello, interpretato e a tipizzazione dinamica, rilasciato pubblicamente nel 1991 da Guido van Rossum. È noto per la sintassi chiara e leggibile, la ricca libreria standard e la facilità di prototipazione rapida. È oggi uno dei linguaggi più usati al mondo, tanto nello sviluppo web quanto nella data science e nell'automazione.

Per il server di DiceRoll, Python è stato scelto per diverse ragioni: la libreria standard include `asyncio` per la programmazione asincrona, la libreria `WebSockets` di terze parti è matura e ben documentata, e la semplicità del linguaggio ha permesso di mantenere il server compatto — poche centinaia di righe — e facilmente leggibile e modificabile.

`Asyncio` è il modulo della libreria standard di Python per la programmazione asincrona basata su coroutine e loop degli eventi. Il suo funzionamento è fondamentalmente diverso dal `multithreading` tradizionale: invece di lanciare un thread separato per ogni connessione (con tutti i problemi di sincronizzazione che ne derivano), `asyncio` gestisce più connessioni all'interno di un singolo thread, sospendendo le coroutine in attesa di I/O e riprendendo la loro esecuzione quando i dati sono disponibili. Questo modello è particolarmente efficiente per applicazioni di rete dove la maggior parte del tempo viene spesa in attesa di messaggi.

 **In breve:** Per un server di chat, il tempo di elaborazione effettiva è minimo: la maggior parte del tempo viene trascorsa ad aspettare che i giocatori inviino messaggi. `asyncio` sfrutta questi momenti di attesa per gestire le altre connessioni, rendendo possibile servire più utenti contemporaneamente con un singolo processo, senza la complessità del `multithreading`.

3.2 Il protocollo WebSocket

WebSocket (standardizzato nella RFC 6455) è un protocollo di comunicazione che nasce come estensione di HTTP ma se ne distacca profondamente nel comportamento. La connessione inizia con un handshake HTTP (l'Upgrade request), al termine del quale il protocollo viene 'promosso' a WebSocket: da quel momento la connessione TCP rimane aperta e sia il client che il server possono inviare messaggi in qualsiasi momento, senza attendere una richiesta.

Questo modello full-duplex è essenziale per una chat in tempo reale: con il tradizionale HTTP, il client dovrebbe interrogare il server continuamente (polling) per sapere se ci sono nuovi messaggi, con un overhead significativo. WebSocket elimina completamente questo problema, consentendo al server di spingere i messaggi ai client nel momento in cui sono disponibili.

 **In breve:** La differenza fondamentale rispetto a HTTP è che con WebSocket la connessione rimane aperta e il server può inviare messaggi ai client in qualsiasi momento, senza aspettare che siano loro a chiedere. Per una chat in tempo reale questo è essenziale: con HTTP si dovrebbe interrogare il server continuamente per sapere se ci sono novità, con un notevole spreco di risorse.

In DiceRoll, tutti i messaggi scambiati tra client e server sono oggetti JSON. Il campo `type` indica il tipo di operazione, e il payload varia di conseguenza. I tipi gestiti sono: `create` (crea

una nuova stanza protetta da password), join (accede a una stanza esistente), message (distribuisce un messaggio — testo o immagine codificata in base64 — a tutti i partecipanti), leave (abbandona la stanza) e ping/pong (heartbeat automatico che permette di tenere in vita la connessione).

3.3 Architettura delle stanze

Il server organizza i client in stanze, identificate dalla password scelta dal primo giocatore che la crea. Password diverse corrispondono a stanze distinte e completamente isolate: i messaggi di una stanza non raggiungono mai i partecipanti di un'altra.

Oltre alla porta WebSocket 8765, il server espone un semplice endpoint HTTP sulla porta 8766 che risponde con il proprio indirizzo IP. Questo endpoint è usato dal client per la funzione di Auto-detect: cliccando il pulsante apposito nella tab Party, l'applicazione interroga automaticamente il server e compila il campo IP senza che l'utente debba cercarlo manualmente.

In caso di disconnessione improvvisa, il client tenta automaticamente la riconnessione fino a tre volte, con un intervallo crescente tra un tentativo e il successivo. Il server notifica tutti i partecipanti della stanza quando un utente si disconnette o si riconnette, mantenendo sempre aggiornato lo stato della sessione.

💡 In breve: Ogni stanza è completamente isolata dalle altre: la password non è solo un controllo di accesso, ma anche il criterio con cui il server smista i messaggi. L'auto-rilevamento dell'IP evita che i giocatori debbano cercarlo manualmente. In caso di disconnessione, il client riprova automaticamente più volte con intervalli crescenti, così da non sovraccaricare il server e dare alla rete il tempo di stabilizzarsi.

```
=====
SERVER CHAT D&D - AVVIATO
=====
WebSocket in ascolto su: 0.0.0.0:8765
IP locale del server: 192.168.1.52
Info endpoint: http://192.168.1.52:8766/info

I client devono connettersi a: 192.168.1.52
Nel campo 'IP Server' inserire: 192.168.1.52
=====
2026-05-04 17:01:26,213 - INFO - server listening on 0.0.0.0:8765
2026-05-04 17:01:55,635 - INFO - connection open
2026-05-04 17:01:55,636 - INFO - Nuova connessione da 192.168.1.52
2026-05-04 17:01:55,642 - INFO - Ricevuto: create da 192.168.1.52
2026-05-04 17:01:55,642 - INFO - Stanza creata: 'password' (ID: e8c9f458) da Thomas (Jason)
2026-05-04 17:01:55,643 - INFO - Stanza 'password' creata da Thomas (Jason)
2026-05-04 17:02:01,284 - INFO - Ricevuto: message da 192.168.1.52
2026-05-04 17:02:20,647 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:02:36,094 - INFO - Ricevuto: message da 192.168.1.52
2026-05-04 17:02:45,644 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:03:10,651 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:03:35,646 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:03:53,113 - INFO - Ricevuto: message da 192.168.1.52
2026-05-04 17:03:59,143 - INFO - Ricevuto: message da 192.168.1.52
2026-05-04 17:04:00,645 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:04:25,642 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:04:50,643 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:04:56,214 - INFO - Ricevuto: message da 192.168.1.52
2026-05-04 17:05:15,648 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:05:41,128 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:06:06,134 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:06:31,133 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:06:56,133 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:07:21,141 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:07:46,142 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:08:11,128 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:08:36,130 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:09:01,134 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:09:26,127 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:09:51,128 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:10:16,142 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:10:40,648 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:11:05,653 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:11:30,648 - INFO - Ricevuto: ping da 192.168.1.52
2026-05-04 17:11:36,678 - INFO - Ricevuto: message da 192.168.1.52
```

Fig. 4 — Architettura delle stanze WebSocket: ogni sessione è isolata e identificata dalla password. Ogni azione è visibile nel log, che viene svuotato solamente alla chiusura del server.

4. Funzionalità principali

In questa sezione vengono descritte in dettaglio le principali aree funzionali dell'applicazione, con particolare attenzione alle scelte implementative più significative.

4.1 Scheda del personaggio e combattimento

La scheda del personaggio è la sezione centrale di DiceRoll e replica fedelmente le meccaniche di D&D 5e. Le sei caratteristiche principali (Forza, Destrezza, Costituzione, Intelligenza, Saggezza, Carisma) sono la base di tutti i calcoli: il modificatore di ogni caratteristica viene calcolato automaticamente con la formula $\text{standard floor}((\text{valore} - 10) / 2)$ e propagato a tutte le sezioni che ne dipendono — abilità, tiri salvezza, iniziativa, bonus agli attacchi con incantesimi.

Le 18 abilità ufficiali di D&D 5e sono tracciate con un doppio stato di competenza (competenza semplice e maestria doppia), che influenza il calcolo del modificatore finale. Il bonus di competenza, basato sul livello del personaggio, viene applicato automaticamente a tutte le abilità in cui il personaggio è competente.

Il sistema di combattimento include la gestione dei Punti Ferita (attuali, massimi e temporanei) con una barra di visualizzazione colorata che cambia tonalità al calare della salute, la Classe Armatura base e temporanea, i Dadi Vita con selezione del tipo, e i Tiri Salvezza contro la Morte con tracciamento visuale dei tre successi e tre fallimenti.

4.2 Mappe e marcatori

La sezione Mappe permette di caricare immagini in formato PNG, JPG o WebP da usare come mappe durante la sessione. L'utente può gestire più mappe contemporaneamente, navigando tra di esse tramite un pannello laterale.

Ogni mappa supporta due modalità di interazione. In modalità Visuale, l'utente può consultare la mappa con i marcatori già posizionati. In modalità Piazzamento, un click sull'immagine aggiunge un nuovo marcatore nella posizione selezionata. Il sistema di marcatori prevede sei categorie con colori distinti: Main Quest (giallo), Personal Quest (viola), Sub Quest (verde), Shop (blu), Boss (rosso) e Item (azzurro).

Ogni marcatore può essere rinominato tramite una modale dedicata o eliminato. La lista completa dei marcatori sulla mappa corrente è visualizzata in un pannello laterale, che funge anche da legenda per i tipi disponibili. Le coordinate dei marcatori vengono normalizzate rispetto alle dimensioni dell'immagine, così da rimanere corrette indipendentemente dallo zoom applicato alla visualizzazione.

4.3 Pannello Dungeon Master

Il pannello DM è progettato per il Dungeon Master e contiene funzionalità che non servono al giocatore normale. Include una sezione per le statistiche dei PNG (personaggi non giocanti) e nemici, con le sei caratteristiche, i Punti Ferita, la Classe Armatura, l'Iniziativa e il Bonus Competenza — tutto indipendente dalla scheda del giocatore.

Il calcolatore dadi avanzato supporta una sintassi flessibile: 2d6+4 tira due dadi da sei e somma quattro, 3d8 tira tre dadi da otto mostrandone ogni risultato separatamente, d20 tira un singolo dado da venti. Sono disponibili pulsanti rapidi per tutti i dadi standard. I risultati vengono mostrati in un log visuale consultabile durante la sessione.

La galleria immagini permette al DM di caricare illustrazioni di mostri, scene di battaglia o ambientazioni da mostrare ai giocatori al momento opportuno. Gli appunti riservati al DM sono separati da quelli del giocatore e non vengono mostrati nelle sezioni pubbliche.

4.4 Party Chat

La Party Chat è la funzionalità multiplayer di DiceRoll che permette a più giocatori sulla stessa rete locale di comunicare in tempo reale senza dover aprire app esterne. Il primo giocatore crea una stanza con una password; gli altri si uniscono inserendo la stessa password e un nickname. La chat supporta messaggi testuali, condivisione di immagini (PNG, JPG, WebP fino a 10 MB, codificate in base64 e trasmesse nel messaggio WebSocket) e tiri di dadi tramite il comando /roll con sintassi standard (es. /roll 2d6+3), il cui risultato viene mostrato a tutti i partecipanti.

L'interfaccia mostra in tempo reale lo stato della connessione (Non connesso, Connessione in corso, Connesso) e notifica gli utenti quando un partecipante si unisce o abbandona la stanza. Lo sfondo della chat è personalizzabile e viene salvato sia nel file JSON della scheda che in localStorage.

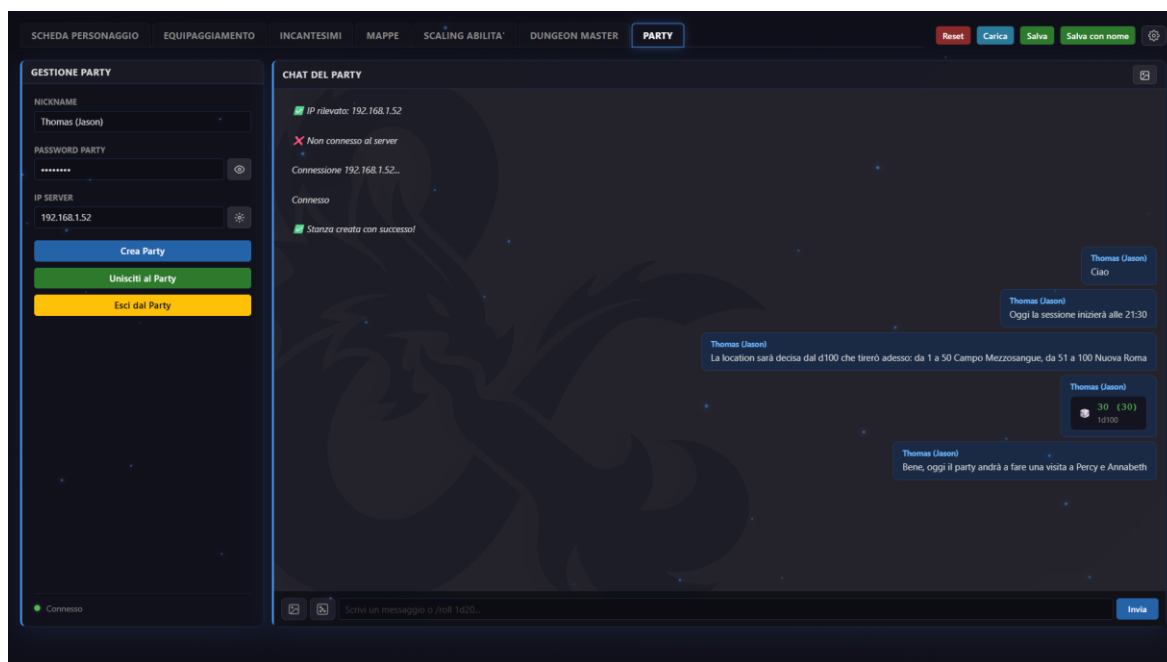


Fig. 5 — Interfaccia della Party Chat con tema di default e stato della connessione in tempo reale.

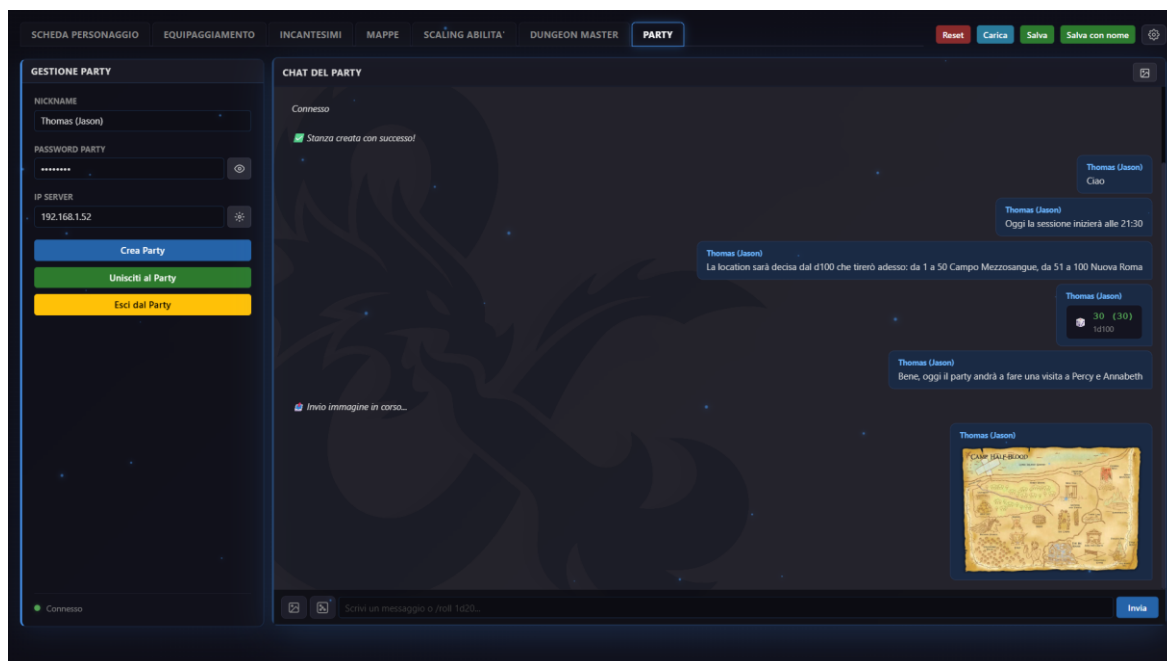


Fig. 6 — Esempio di Party Chat con messaggi, tiri di dadi e immagini.

5. Sviluppo e strumenti

Lo sviluppo di DiceRoll è stato affrontato seguendo un approccio incrementale: prima le funzionalità di base della scheda del personaggio, poi i moduli più complessi come le mappe e la chat, e infine il packaging con Electron. In ogni fase è stato usato Git per il controllo versione.

5.1 Git e controllo versione

Git è il sistema di controllo versione distribuito più diffuso al mondo. Consente di registrare ogni modifica apportata al codice in forma di commit, navigare lo storico delle versioni, lavorare su funzionalità parallele tramite branch e tornare a uno stato precedente in caso di regressioni. Per DiceRoll, Git ha permesso di sperimentare nuove funzionalità — come il sistema delle mappe o la gestione dei temi — su branch separati, integrando le modifiche nel ramo principale solo a sviluppo completato e verificato.

5.2 Electron Forge

Electron Forge è lo strumento ufficiale per il packaging e la distribuzione di applicazioni Electron. Legge la configurazione da `forge.config.js` e automatizza la compilazione del progetto, raccogliendo il codice sorgente, le dipendenze e il runtime Electron in un unico eseguibile distribuibile. Il risultato è un file `.exe` che l'utente finale può avviare direttamente, senza installare Node.js o altri strumenti.

5.3 Difficoltà riscontrate

Nel corso dello sviluppo sono emerse diverse sfide tecniche che hanno richiesto approfondimento e soluzioni ad hoc:

- **Gestione dello stato in Vanilla JS:** senza un framework reattivo, ogni aggiornamento del DOM deve essere gestito esplicitamente. È stato necessario definire una struttura chiara dello stato dell'applicazione e funzioni dedicate per sincronizzare l'interfaccia con i dati ogni volta che un valore cambia;
- **File System Access API in Electron:** le API moderne del browser si comportano in modo leggermente diverso nell'ambiente Electron rispetto a un browser standard. È stato necessario configurare correttamente i permessi nel main process per consentire l'accesso al filesystem;
- **Coordinate marcatori su immagini scalate:** il posizionamento dei marcatori sulle mappe ha richiesto un sistema di coordinate normalizzate, per garantire che la posizione rimanga corretta indipendentemente dallo zoom o dalle dimensioni della finestra al momento del salvataggio;
- **Stabilità della connessione WebSocket:** le connessioni su rete locale possono cadere per vari motivi. È stato implementato un sistema di heartbeat (ping/pong) e un meccanismo di riconnessione automatica con retry progressivo, per rendere la Party Chat robusta anche in condizioni di rete non ideali.

6. Tecnologie utilizzate

Componente	Tecnologia
Struttura pagina	HTML5
Stile e layout	CSS3, Bootstrap 5.3
Logica client	JavaScript ES6+ (Vanilla JS)
Runtime desktop	Electron
Packaging / build	Electron Forge
Salvataggio file	File System Access API / Blob download
Persistenza locale	localStorage (con salvataggio automatico)
Internazionalizzazione	Sistema i18n custom (ita.js / eng.js)
Server chat	Python 3, asyncio
Libreria WebSocket	WebSockets (PyPI)
Protocollo real-time	WebSocket RFC 6455
Controllo versione	Git

6.1 Motivazione dietro alle tecnologie utilizzate

Tecnologia	Perché è stata scelta?
HTML5	Struttura dell'interfaccia
CSS3	Stile grafico e temi personalizzabili
Bootstrap	Sviluppo rapido e layout responsive
JavaScript ES6+ (Vanilla JS)	Logica applicativa e gestione del DOM
Electron + Electron Forge	Applicazione desktop eseguibile
File JSON	Serializzazione e scambio dati
File System Access API	Salvataggio diretto sul filesystem locale
LocalStorage	Backup automatico e persistenza locale
Python	Sviluppo del server multiplayer
Asyncio	Gestione asincrona delle connessioni
WebSocket + WebSockets (Python)	Comunicazione multiplayer in tempo reale
Git	Controllo versione e gestione sviluppo
Architettura offline-first	Privacy, indipendenza dal cloud e uso offline

7. Conclusioni

DiceRoll è nato come risposta a un'esigenza concreta — avere una scheda del personaggio digitale e più flessibile rispetto a quella ufficiale da utilizzare durante le sessioni di D&D — e si è trasformato nel corso dello sviluppo in un progetto decisamente più articolato, che tocca frontend, backend, protocolli di rete e distribuzione di applicazioni desktop.

Dal punto di vista tecnico, il progetto ha permesso di approfondire in modo pratico numerose tecnologie: la programmazione frontend in JavaScript puro, la gestione di file con API moderne del browser, l'internazionalizzazione di un'interfaccia complessa, il protocollo WebSocket e la programmazione asincrona in Python, e il processo di packaging di un'applicazione Electron. Ogni area ha presentato le sue sfide specifiche, che hanno richiesto ricerca, sperimentazione e soluzioni costruite su misura.

La scelta di non usare framework pesanti come React o Vue — pur richiedendo uno sforzo maggiore nella gestione dello stato e del DOM — ha reso il codice più diretto e comprensibile, e ha garantito una comprensione profonda dei meccanismi che normalmente vengono nascosti dalle astrazioni dei framework. Allo stesso modo, la scelta di Python per il server ha dimostrato come un linguaggio semplice e una libreria standard ben progettata siano sufficienti per implementare un backend funzionale e robusto.

7.1 Possibili Sviluppi Futuri

Guardando avanti, gli sviluppi più interessanti che si potrebbero esplorare includono la condivisione in tempo reale della scheda del personaggio e delle relative statistiche tra tutti i giocatori del party, il supporto a sistemi di gioco diversi da D&D 5e — come “Seventh Sea” oppure “Sì, Oscuro Signore” —, e la possibilità di ospitare il server su Internet tramite tunnel sicuro per permettere sessioni di gioco a distanza.

Una delle evoluzioni più significative riguarderebbe il passaggio da un modello puramente locale a un sistema con autenticazione utente e persistenza remota delle schede. Attualmente ogni scheda viene salvata come file JSON sul filesystem locale dell'utente, una scelta coerente con la natura offline-first del progetto. Tuttavia, questa struttura si presta naturalmente a una migrazione verso MongoDB: essendo un database documentale che archivia i dati in formato BSON — strutturalmente analogo a JSON — ogni scheda potrebbe essere salvata come documento nella propria collezione senza richiedere modifiche alla struttura dati esistente. Un sistema di login permetterebbe a ciascun utente di accedere alle proprie schede da qualsiasi dispositivo, sincronizzare automaticamente le modifiche durante la sessione e condividere la scheda in sola lettura con gli altri giocatori del party. Il server Python attuale potrebbe essere esteso con un layer REST o GraphQL per gestire l'autenticazione tramite JWT e le operazioni CRUD sulle schede, mantenendo inalterato il modulo WebSocket già esistente per la chat in tempo reale.